Government Data APIs

Week 11 · APIs module · Textbook Chapter 11

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

October 26, 28 & 30, 2026

This week at a glance

We query the data the government is *required* to publish — and combine it into stories no single dataset can tell.

Monday | Concepts & API keys

- Why government data is open; managing keys like a professional

Wednesday | Notebook lab

- Query the Census, FRED, and FEC — then join two of them

Friday | Textbook revisions

- Propose & peer-review pull requests improving Chapter 11

Companion notebook

`ch-11-government.ipynb`

Get your keys now

Three free keys — **Census**, **FRED**, **FEC**. Approval is fast, but no code below runs without them.

1. Monday · Concepts

Public data by mandate

Government APIs exist because of democratic commitments to **transparency** — not business strategy:

- The **Census Bureau** is constitutionally mandated to count the population.
- The **Federal Reserve** is required to publish economic data.
- The **FEC** must disclose campaign-finance records.

They embody the **openness** value from Ch. 3, *The Post-API Age*: public data, freely accessible by mandate rather than platform goodwill.

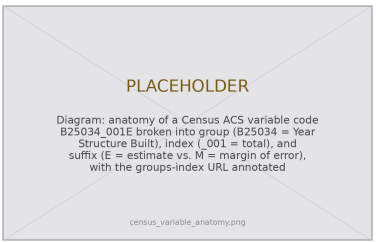
Stable, but less polished

Agencies don't pivot business models or get acquired — so these APIs are unusually **stable**. The tradeoff: dense docs, cryptic variable names, unhelpful errors. Navigating that *is* the skill.

The challenge is navigation, not access

The Census API is free and well-maintained. The hard part is **finding the right variable** among thousands.

- Start at the **groups index**; search broad categories (housing, income, “commut”).
- Drill into a group’s variables and read the labels — population denominators differ.
- Codes shift across survey years: *learn to read them*, don’t memorize them.
- Always **sanity-check**: Boulder median income near $70,000, not 7 or 70,000,000.



Read the code: group, index, `E`=estimate / `M`=margin of error.

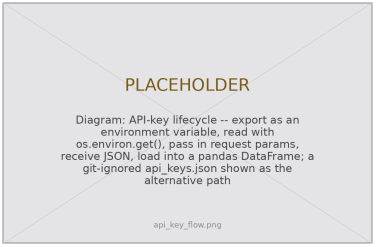
Manage your keys like a professional

A **key** identifies your app, tracks usage, and enforces rate limits. **Never hardcode one** — share the notebook and the key goes with it.

Two safe patterns, both ending at the same variables:

- **Environment variables** — `export CENSUS_API_KEY=...`, then `os.environ.get()`.
- **A JSON config** — `api_keys.json`, added to `.gitignore` *before your first commit*.

Because both yield `census_key` / `fred_key` / `fec_key`, downstream code is identical.



In the notebook

This week's companion notebook works three agencies, then combines two:

- **Census —** navigate the ACS catalog; housing age for Boulder; compare cities side by side.
- **FRED —** income time series; server-side transformations; Denver vs. national inflation.
- **FEC —** search a candidate; pull their campaign-finance totals.
- **Integrate —** join Census housing vintage with FRED mortgage rates in one figure.

Public interest (Ch. 3)

Open data is a mandate, not a guarantee. Datasets get withdrawn and APIs deprecated; political pressure can shape what is collected and how. The **fragility** of public data infrastructure is itself a public-interest concern.

2. Wednesday · Notebook Lab

Load every key once — and fail fast

Read the keys at the top; stop immediately with a helpful message if one is missing:

```
import os, requests, pandas as pd

# Set each in your shell first -- never hardcode a key in a notebook:
# export CENSUS_API_KEY="your-key-here"
census_key = os.environ.get("CENSUS_API_KEY")
fred_key = os.environ.get("FRED_API_KEY")
fec_key = os.environ.get("FEC_API_KEY")

for name, key in [("CENSUS_API_KEY", census_key),
                  ("FRED_API_KEY", fred_key), ("FEC_API_KEY", fec_key)]:
    if not key:
        raise ValueError(f"{name} not set -- get a free key first.")
```

Census — from the catalog to a real place

Geographies use FIPS codes, which are **strings** — Colorado is `08`, Boulder is place `07850`. Keep the leading zeros:

```
census_url = "https://api.census.gov/data/2022/acs/acs5"

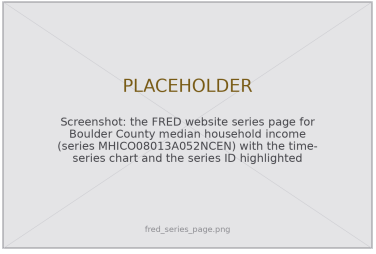
# Housing age for Boulder, CO -- browse groups at .../acs5/groups.html
params = {
    "get": "NAME,B25034_001E,B25034_002E,B25034_003E",
    "for": "place:07850", # FIPS are STRINGS -- do not strip 0s
    "in": "state:08",
    "key": census_key,
}
data = requests.get(census_url, params=params).json()
df = pd.DataFrame(data[1:], columns=data[0]) # row 0 is the header
```

FRED — let the server do the math

Ask FRED for a **transformation** and it returns the result — here, year-over-year percent change:

```
fred_url = ("https://api.stlouisfed.org"
            "/fred/series/observations")

params = {
    "series_id": "CUURS49BSA0", # Denver CPI
    "api_key": fred_key,
    "file_type": "json",
    "units": "pc1", # % change vs. 1yr ago
    "observation_start": "2019-01-01",
}
obs = requests.get(fred_url,
        params=params).json()["observations"]
df = pd.DataFrame(obs)
```



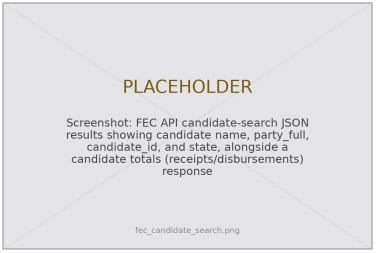
Find series IDs on the FRED website, then copy them *exactly*.

FEC — money in politics in two steps

Search by name to get a `candidate_id`, then pull that candidate's financial totals:

```
fec_url = "https://api.open.fec.gov/v1/candidates/search/"
params = {"q": "Biden", "office": "P", # P/S/H
          "election_year": 2024,
          "api_key": fec_key, "per_page": 5}
for c in requests.get(fec_url,
        params=params).json()["results"]:
    print(c["name"], c["party_full"], c["candidate_id"])

cid = "P80000722" # a "P" id = presidential candidate
url = f"https://api.open.fec.gov/v1/candidate/{cid}/totals/"
tot = requests.get(url, params={"api_key": fec_key,
                    "election_year": 2024}).json()
```



IDs encode the office: `P`/`S`/`H` = candidate, `C` = committee.

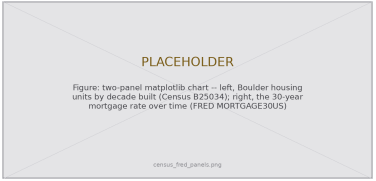
Combine two agencies — and mind the sentinels

The payoff is integration. But guard Census **sentinel values** — suppressed cells return as `-666666666`, not null:

```
def census_int(v): # NEVER trust a bare int()
    if v is None or str(v).startswith("-6666"):
        return None
    return int(v)

housing = requests.get(census_url,
            params=housing_params).json()
counts = [census_int(housing[1][i])
          for i in range(2, 8)] # by decade

# FRED: 30-yr mortgage rate = the economic "why"
rate = requests.get(fred_url,
          params=mortgage_params).json()
df_rate = pd.DataFrame(rate["observations"])
```



Census says *what* the housing stock is; FRED says *why*.

Lab: work these, then show-and-tell

Work in pairs. Do these exercises from the chapter:

1. Get one Census variable group for 5 or more geographies. Make a visualization that compares them.
2. Plot two related FRED series for your area on the same axes. Explain what they show.
3. Get the FEC contributions for a local congressional race.
4. Combine data from two government APIs in one analysis.
5. Get one Census variable for all Colorado counties with `county:*`. Make a bar chart of the 20 highest values.
6. **5617**: read Lazer et al. (2020). Link two APIs. Write a limitations section about measurement and coverage.

Show-and-Tell

Bring one of these to class:

- a bug that you could not solve
- a number that surprised you
- a question for a research design

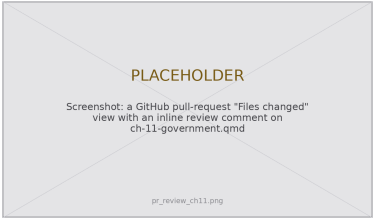
3. Friday · Textbook Revisions

Improving Chapter 11 together

Today we make the book better. Each of you opens a **pull request** against `ch-11-government.qmd` and reviews a classmate's.

The workflow (from Week 1):

1. Branch / fork the book repo
2. Edit the chapter; commit with a clear message
3. Open a PR describing *what* and *why*
4. Review two peers' PRs: kind, specific, actionable



A revision menu for Chapter 11

High-value targets — pick one and scope it to a single PR:

- **Test the code against live APIs.** Series IDs, endpoints, and election years drift; keys expire. Run the notebook, report what broke, and fix it.
- **Close a gap in "Common Issues to Debug."** Add a worked `-666666666` sentinel-value example, or a rate-limit / retry snippet (FRED ≈120 req/min; FEC 1,000 req/hr).
- **Add a figure.** An annotated variable-code diagram, or the key → request → JSON → DataFrame pipeline.
- **Strengthen a thin exercise.** The six exercises are open-ended — add expected output, a rubric, or a starter cell so they self-check.
- **Extend "Further Reading."** A data-journalism example tied to the **Public Interest** callout — a documented withdrawn dataset or deprecated API.

A good revision, and a good review

A strong PR

- One focused change, clearly titled
- Explains the problem it fixes
- Builds (`quarto render`) without errors
- Matches the book's voice and notation
- Discloses any AI assistance

A strong review

- Runs / reads the change, not just skims
- Names specifics (line, claim, code)
- Separates "must fix" from "nice to have"
- Is generous — assume good faith
- Ends with a clear verdict

Next week: **Social & media platform APIs —** OAuth, Mastodon, and Bluesky in the post-API age.

Key takeaways

1. Government APIs serve **transparency mandates** — valuable, reliable, and unusually stable data.
2. The core skill is **navigating catalogs**, not memorizing codes that change every survey year.
3. **Guard your keys**: environment variables or a git-ignored config — never hardcoded.
4. **Combine sources** for stories no single dataset tells: Census says *what*, FRED says *why*.
5. Mind the **institutional context** — using open data well is a form of civic engagement.